

PRACTICAL - 6

AIM : Implementation of chain matrix multiplication using dynamic programming.

ALGORITHM :

Algorithm :-

Input : Matrix dimensions $P[0], P[1] \dots P[n]$
Output : minimum number of scalar multiplication.

Step-1 :- Start
Step-2 :- Read n and matrix dimension $P[0]$ to $P[n]$
Step-3 :- Set $c[i][j] = 0$ for $i=1$ to n
Step-4 :- Repeat for $d=1$ to $n-1$
Step-5 :- Repeat for $i=1$ to $n-d$
Step-6 :- Set $j=i+d$
Step-7 :- Set $min = 32767$
Step-8 :- Repeat for $k=i$ to $j-1$
Step-9 :- calculate.
 $q = c[i][k] + c[k+1][j] + P[i-1] \times P[k] \times P[j]$
Step-10 :- if $q < min$, set $min = q$
Step-11 :- Set $c[i][j] = min$.
Step-12 :- Repeat step 8 to 11 for all
Step-13 :- Repeat step 3 to 12 for all
Step-14 :- Print $c[i][n]$
Step-15 :- stop.

PROGRAM :

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

int main() {

    int n;

    // Step 1: Number of matrices
    cout << "Enter the number of matrices: ";
    cin >> n;
    // p contains n + 1 dimensions
```

```
vector<int> p(n + 1);

cout << "Enter " << n + 1 << " dimensions: ";
for (int i = 0; i <= n; i++) {
    cin >> p[i];
}

// DP table
vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));

// Step 2: Chain length
for (int len = 2; len <= n; len++) {

    // Step 3: Starting matrix
    for (int i = 1; i <= n - len + 1; i++) {

        int j = i + len - 1;

        dp[i][j] = INT_MAX;

        // Step 4: Try every possible split
        for (int k = i; k < j; k++) {

            int cost = dp[i][k]
                + dp[k + 1][j]
                + p[i - 1] * p[k] * p[j];

            if (cost < dp[i][j]) {
                dp[i][j] = cost;
            }
        }
    }
}

// Step 5: Print DP Table
cout << "\nDP Table:\n\n";

// Column headings
cout << "    ";
for (int j = 1; j <= n; j++) {
    cout << "M" << j << "t";
}
cout << endl;
```

```
for (int i = 1; i <= n; i++) {  
  
    cout << "M" << i << "  ";  
  
    for (int j = 1; j <= n; j++) {  
  
        if (i > j) {  
            cout << "-\t";  
        }  
        else {  
            cout << dp[i][j] << "\t";  
        }  
    }  
  
    cout << endl;  
}  
  
// Step 6: Minimum cost  
cout << "\nMinimum number of scalar multiplications = "  
    << dp[1][n] << endl;  
  
return 0;  
}
```

OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB  
> g++ MCM.cpp -o MCM
```

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB  
> ./MCM
```

```
Enter the number of matrices: 4  
Enter 5 dimensions: 5 4 6 2 7
```

DP Table:

	M1	M2	M3	M4
M1	0 120	88	158	
M2	- 0	48	104	
M3	- -	0	84	
M4	- -	-	0	

Minimum number of scalar multiplications = 158



TIME COMPLEXITY :

Time Complexity :-

```

for (int d=1; d<n; d++) {
    for (int i=1; i<=n-d; i++) {
        for (int k=i; k<=n; k++) {
            // ...
        }
    }
}

```

$$T(n) = \sum_{d=1}^{n-1} \sum_{i=1}^{n-d} \sum_{k=i}^n 1$$

$$T(n) = \sum_{d=1}^{n-1} \left(\sum_{i=1}^{n-d} (n-d+1) \right)$$

$$\sum_{d=1}^{n-1} d = \frac{n(n-1)}{2} \quad \sum_{d=1}^{n-1} d^2 = \frac{n(n-1)(2n-1)}{6}$$

$$T(n) = \frac{n(n-1)(n+1)}{6} \quad T(n) = \frac{n^3 - n}{6}$$

$$T(n) = O(n^3) \rightarrow \text{worst case.}$$

Best case :- Even in the best case, the program still executes the same these loops
 $T(n) = \frac{n^2 - n}{6} \quad O(n^3)$

Average case :- loop also executes the same no. of times regardless of the matrix dimension.
 $T(n) = O(n^3)$.

CONCLUSION :

I successfully implemented and understood the Matrix Chain Multiplication problem using the Dynamic Programming approach. I learned how to determine the most efficient order of multiplying a sequence of matrices by minimising the total number of scalar multiplications. By using a cost table and storing previously calculated results, I was able to avoid repeated calculations and improve the efficiency of the solution. This practical helped me understand the application of dynamic programming in solving optimisation problems and improved my understanding of time and space complexity.